Tu primer proyecto con Python y GitHub

MERCADO APRENDE | Erik José Vera Mercado y su hermano | 10 de octubre de 2026

Una aplicación para publicar, buscar y comprar productos de forma simulada. El objetivo es aprender a construir una web y colaborar con GitHub, paso a paso.

Qué ya hice

Creé desde cero la aplicación Flask, la base SQLite, las plantillas HTML, el diseño responsive, los comentarios en español, las pruebas y un workflow de GitHub Actions. Usé el concepto de marketplace de Team Alpha como referencia, sin copiar su código ni modificar el proyecto del equipo.

GitHub: repositorio creado

Tras completar el acceso a GitHub, creé el repositorio privado erickvera610/mercado-aprende desde New repository. Activé README para crear el primer commit. Preparé la subida del código completo por la conexión de GitHub. Esta guía describe también cómo repetir el proceso en tu computador.

Resultado y límites

Incluye registro, login, publicaciones, búsqueda, filtros, detalles y compras simuladas. No cobra dinero ni gestiona envíos. Se ejecuta en tu computador. Guardar código en GitHub no publica el servidor Python en Internet.

01 / Prepara y abre la aplicación

Instala Python 3.12 y Git desde sus sitios oficiales; Visual Studio Code es opcional. En Windows, activa la opción de añadir Python a PATH. Extrae el ZIP y abre PowerShell dentro de la carpeta mercado-aprende.

```
py -3.12 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe app.py
```

La primera línea crea un entorno virtual aislado. La segunda instala Flask y pytest en ese entorno. La tercera inicia el servidor. No hace falta activar el entorno porque usamos directamente su Python.

Abre http://127.0.0.1:5000 en tu navegador. Deja la terminal abierta. Para detener el servidor pulsa Ctrl+C. Para abrirlo de nuevo, ejecuta la tercera línea.

En macOS o Linux

```
python3 -m venv .venv
.venv/bin/python -m pip install -r requirements.txt
.venv/bin/python app.py
```

Primera prueba manual

1. Crea vendedor con una contraseña de práctica de al menos 8 caracteres. 2. Inicia sesión y publica Bicicleta azul, categoría Deportes, estado Usado, precio 125.50 y descripción. 3. Busca el producto. 4. Cierra sesión y crea comprador. 5. Compra con esa segunda cuenta. 6. Revisa Mi cuenta y confirma que desapareció del catálogo disponible.

02 / Entiende cada parte del código

Python ejecuta la lógica; Flask recibe solicitudes; HTML define las páginas; CSS controla el diseño; SQLite guarda los datos. Git registra versiones y GitHub permite guardar y revisar esas versiones con otra persona.

Archivos y responsabilidades

app.py: servidor y reglas del mercado. templates/base.html: navegación y mensajes. index.html: catálogo. auth.html: registro y login. sell.html: publicación. detail.html: detalle y compra. account.html: historial. static/style.css: colores y distribución. tests/test_market.py: pruebas. .github/workflows/tests.yml: automatización.

Lee app.py en este orden

1. Imports y constantes: herramientas y opciones permitidas. 2. create_app: configura y construye Flask. 3. db y tablas: abre SQLite y define users y listings. 4. csrf_token y before_request: cargan el usuario y protegen formularios. 5. login_required: exige sesión. 6. register y login: cuentas. 7. index y sell: productos. 8. detail, buy y account: compras e historial.

Cómo viaja una petición

GET /sell muestra un formulario. POST /sell recibe los campos, valida, inserta una fila y redirige al catálogo. render_template combina datos con HTML. flash muestra mensajes. session recuerda quién inició sesión. buyer vacío significa que el producto está disponible.

La compra usa un UPDATE condicionado: solo cambia buyer si sigue vacío y el vendedor es otra persona. Así evita compras propias y ventas duplicadas. Cada bloque está explicado dentro del código Python.

03 / Comprueba antes de subir

Pruebas automatizadas

Desde la carpeta principal, ejecuta el comando siguiente. Cada prueba utiliza una base temporal independiente; no modifica tus productos.

```
.\.venv\Scripts\python.exe -m pytest -q
```

En macOS/Linux utiliza .venv/bin/python -m pytest -q. Las pruebas cubren el flujo completo, compra propia, compra repetida, precios inválidos, login, logout, CSRF, contraseñas con hash, búsqueda, categorías y escape de HTML.

Las decisiones importantes

Las contraseñas se guardan como hashes. Los parámetros SQL usan ? para separar datos de instrucciones. Decimal convierte precios a centavos enteros. Los campos se validan en el servidor. Los formularios incluyen tokens CSRF. Las plantillas escapan texto.

Qué no debes subir

La base instance/market.db contiene las cuentas locales. .venv guarda dependencias. .env puede contener secretos. Todos están excluidos en .gitignore. Antes de un commit, revisa git status: esos elementos no deben aparecer.

Persistencia y sesión

Los productos sobreviven al reinicio porque están en SQLite. La sesión puede expirar al reiniciar si no configuraste SECRET_KEY en el entorno. Recarga la página e inicia sesión otra vez. Para varios procesos se necesita la misma clave estable, almacenada fuera de Git.

04 / Cómo creé el repositorio y subí el código

En GitHub, inicia sesión como erickvera610. Pulsa + y New repository. Nombre: mercado-aprende. Descripción: Marketplace educativo en Python y Flask con tutorial en español. Elige Private para comenzar. No añadas README, licencia ni .gitignore: ya están preparados. Pulsa Create repository.

Usé New repository para crear mercado-aprende privado y un README inicial. Para subir archivos utilicé la conexión de GitHub: árbol de archivos, commit y actualización de main. Los comandos siguientes enseñan la alternativa con Git local para un repositorio nuevo y vacío; no los ejecutes sobre este repositorio ya inicializado:

```
git init -b main
git status
git add .
git commit -m "Crear marketplace educativo en Flask"
git remote add origin https://github.com/erickvera610/mercado-aprende.git
git push -u origin main
```

init comienza el historial. status muestra cambios. add prepara archivos. commit crea una versión local. remote add conecta el repositorio remoto. push envía la versión. -u conecta main local con main remoto.

Si Git solicita identidad, configura git config user.name "Erik José Vera Mercado" y git config user.email con tu correo noreply exacto de GitHub Settings > Emails. No inventes ese correo. Autentícate por el navegador o administrador de credenciales. No compartas contraseñas ni tokens en chat.

Verifica el resultado

Abre el repositorio y confirma que aparecen los archivos. En Actions busca Python tests y espera el resultado verde. La ejecución local no demuestra que Actions ya haya corrido. GitHub Pages no ejecuta Flask: esta app requiere un servidor Python para alojarse en Internet.

05 / Aprende con ramas y Pull Requests

Cuando quieras compartir el repositorio privado, invita a tu hermano desde Settings > Collaborators. Él acepta y usa su propia cuenta. No se ha enviado ninguna invitación.

Después de crear y subir el repo, tu hermano puede ejecutar:

```
git clone https://github.com/erickvera610/mercado-aprende.git
cd mercado-aprende
git switch -c mejora-texto
```

Pídele cambiar una frase en templates/index.html. Una rama separa su trabajo de main. Después revisa y guarda:

```
git diff
git add templates/index.html
git commit -m "Mejorar texto de bienvenida"
git push -u origin mejora-texto
```

En GitHub, pulsa Compare & pull request. Explica qué cambió y por qué. Tú revisas Files changed y las pruebas. Si todo está correcto, usa Merge pull request. Después ambos actualizan su copia:

```
git switch main
git pull
```

Un commit es una versión; una rama es una línea de trabajo; un Pull Request es una propuesta de integrar esa línea. Un GitHub Project es un tablero de tareas distinto del repositorio. Este proyecto entrega una aplicación en un repositorio; no crea un tablero.

06 / Soluciona problemas y sigue aprendiendo

La app no abre

Comprueba que app.py siga activo y abre 127.0.0.1:5000. Si falta Flask, instala requirements.txt con el Python de .venv. Si el puerto está ocupado, cambia port=5000 en app.py y abre el nuevo puerto. Si dice formulario expirado, recarga e inicia sesión.

Git no sube

not a git repository: estás fuera de la carpeta. origin already exists: revisa git remote -v antes de cambiarlo. push rechazado: no uses force; ejecuta git pull --rebase origin main, resuelve conflictos, vuelve a probar y sube. Falta de acceso: revisa cuenta e invitación.

Siguientes ejercicios

Primero cambia un texto y abre un Pull Request. Luego añade un filtro de condición con su prueba. Después considera editar publicaciones e incorporar imágenes. Una mejora pequeña por rama hace más fácil revisar y aprender.

Ejecutar en GitHub Codespaces

En el repo abre Code > Codespaces > Create codespace on main. La configuración instala las dependencias y arranca Flask. Abre Ports y el puerto 5000, conservando su visibilidad privada. Si hace falta, ejecuta bash .devcontainer/start.sh. Detén el Codespace al terminar: consume la cuota de tu cuenta y puede tener costes al superarla. No es hosting permanente.

Antes de convertirlo en tienda real

Hacen falta HTTPS, un servidor de producción, clave estable, límites de intentos de login, recuperación de cuentas, migraciones, backups, pagos, privacidad y una revisión de seguridad. No publiques el servidor de desarrollo como una tienda comercial.

Resumen de entrega

Aplicación original documentada + pruebas + workflow + tutorial editable y PDF. El repositorio privado está creado. Incluí configuración de Codespaces para instalar dependencias e iniciar la aplicación en el navegador. No se modificaron los repositorios existentes ni se copiaron archivos del equipo.